

# Power forward curves toolkit

Matteo Gardini \*

July 2, 2026

## Abstract

This article aims at showing how to construct the power forward curves. Based on our experience in the field we discuss about possible approaches highlighting strengths and weaknesses.

**Keywords:** Power forward curves, non arbitrage constraints, splines.

## 1 Introduction

Electricity is usually labeled as “commodity”, although it is not storable. In such markets some contracts prescribe a physical delivery (namely the actual electricity is physically delivered) others prescribe a financial delivery: such markets are usually settled in cash against a given reference price, typically the spot price. The latter contracts are settled in cash against the system prices over a specified delivery period. For this reason, they are in fact swap contracts, exchanging a floating spot price against a fixed one. However, in financial jargon these contracts are called *electricity futures*.

Such contracts appears in blocks, each characterized by a *time to delivery* and by a *delivery period*. For example, the month ahead product (M1) is quoted representing the price of electricity for each hour of the next month (base-load contract). Similarly, the M2, M3, M4, and subsequent contracts correspond to deliveries two, three, and four months ahead, respectively. Each month, the nearest monthly contract expires and is delisted, while a new contract is introduced. The same structure applies to longer delivery periods, such as quarters (Q1, Q2, ...) and years (Y1, Y2, ...).

Clearly, in this context, *no-arbitrage constraints* must hold. For example, if contracts for three consecutive months and the corresponding quarterly contract are simultaneously listed, the value of the quarterly contract must equal the average of the prices of the three monthly contracts. For a good overview of the market refer to Benth et al. [1, Chapters 1-2].

The collection of the quoted forward (swaps) products is known as *forward curve* and the key driver for swap prices is the electricity spot prices. Now we introduce some notation following Benth et al. [1, Chapter 1]. The spot price is denoted by  $S = \{S(t); t \geq 0\}$  and by  $f(t, \tau)$  the forward price at time  $t \geq \tau$  of entry the contract: the payoff at delivery time

---

\*Eni Plenitude, Via Lorenzini 4, 20136, Milan, Italy, email [matteo.gardini@eniplenitude.com](mailto:matteo.gardini@eniplenitude.com), [matteo.gardini88@gmail.com](mailto:matteo.gardini88@gmail.com)

$\tau$  is  $S(\tau) - f(t, \tau)$ .

Since entering a forward contract has no cost we require (from the theory of mathematical finance):

$$e^{-r(\tau-t)} \mathbb{E}^{\mathbb{Q}} [S(\tau) - f(t, \tau) | \mathcal{F}_t] = 0.$$

and hence:

$$f(t, \tau) = \mathbb{E}^{\mathbb{Q}} [S(\tau) | \mathcal{F}_t].$$

We denote the electricity future contract with delivery over time interval  $[\tau^s, \tau^e]$  at  $t \leq \tau^s$  by  $F(t, \tau^s, \tau^e)$  quoted in EUR/MWh. A useful relation is provided by Benth et al. [1, Proposition 4.1]:

$$F(t, \tau^s, \tau^e) = \int_{\tau^s}^{\tau^e} w(u, \tau^s, \tau^e) f(t, u) du \quad (1)$$

where  $w(u, \tau^s, \tau^e)$  is a weighing function such that  $\int_{\tau^s}^{\tau^e} w(u, s, t) du = 1$ . Holding a swap contract can be considered as holding a (weighted) continuous stream of forwards. The goal of the next chapter is to construct the curve  $f(t, u)$  satisfying non-arbitrage constraints like Equation (1).

## 2 The hourly power forward curve

Representing forward prices through a continuous term structure is essential for modelling forward price dynamics and for marking over-the-counter (OTC) contracts to market. In electricity markets, a substantial volume of financial contracts is traded OTC, and many of these contracts have delivery periods that do not coincide with those of the standardized products listed on the exchange. Consequently, market participants must infer the market value of such contracts by combining the prices of the available exchange-traded products to construct a consistent hourly forward curve (or even with fine granularity). The following discussion follows ideas presented in Benth et al. [1, Chapter 7].

### 2.1 Market products and notation

Assume that at time  $t$  we observe  $m$  swap contracts  $\{F(t, \tau_i^s, \tau_i^e)\}_{i=1}^m$  as in Figure 1. In this case no complete overlapping products are present: if so, the totally overlapped products must be removed before proceeding since totally overlapped products does not add any market information. We assume that  $t = 0$  and we simplify the notation writing:

$$\begin{aligned} F(\tau^s, \tau^e) &= F(0, \tau^s, \tau^e), \\ f(u) &= f(0, u) \end{aligned}$$

We use as weighting function:

$$w(u, s, t) = \frac{1}{t - s}.$$

As mentioned before, we have  $m$  swap contracts, each with delivery period  $[\tau_i^s, \tau_i^e]$ . We list them in  $S_p = \{(\tau_1^s, \tau_1^e), \dots, (\tau_m^s, \tau_m^e)\}$  and we create a new list of dates containing the unique values of  $S_p$ :  $\{\tau_0, \dots, \tau_n\}$ . We have  $n+1$  points in the grid and hence  $n$  intervals.

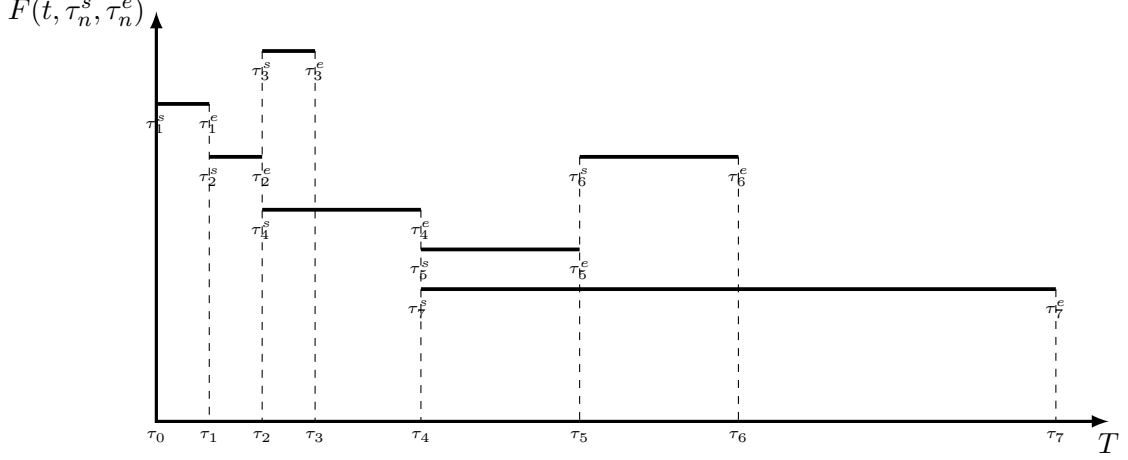


Figure 1: Quoted futures product  $F(t, \tau_n^s, \tau_n^e)$ .

Now we need a functional form for the hourly forward curve  $f(u)$ : several assumption can be made for this curve, depending on the needs and on the situations, but typically

$$f(u) = g(\Lambda(u), \epsilon(u)),$$

where  $\Lambda(u)$  represents a general seasonality function,  $\epsilon(u)$  encodes the forward market information and  $g$  is a general function. Essentially, the forward curve is a mix between historical seasonality and market expectations about the future. In the next section we detail the construction of the hourly forward curve.

## 2.2 Construction of the forward curve

A first assumption for  $g$  is  $g(x, y) = x + y$  and hence:

$$f(u) = \Lambda(u) + \epsilon(u), \tag{2}$$

where  $\Lambda(u)$  represent the seasonality, whereas  $\epsilon(u)$  is the adjustment function capturing the forward's curve deviation from seasonality and incorporates the information provided by the forward market: it can be interpreted as risk premium. In Section 2.4 we will consider  $g(x, y) = xy$ , leading to  $f(u) = \Lambda(u)\epsilon(u)$ , namely a multiplicative forward curve.

We assume that functional form for  $\epsilon(u)$  is a piece-wise polynomial of fourth degree whereas the form of  $\Lambda(u)$  is free and will be specified later on.

$$\epsilon(u) = \begin{cases} a_1 u^4 + b_1 u^3 + c_1 u^2 + d_1 u + e_1, & u \in [\tau_0, \tau_1] \\ \dots & \\ a_n u^4 + b_n u^3 + c_n u^2 + d_n u + e_n, & u \in [\tau_{n-1}, \tau_n] \end{cases}$$

As mentioned above, there are  $n$  intervals, 5 parameters per interval ( $a_i, b_i, c_i, d_i, e_i$  for  $i = 1, \dots, n$ ) and hence a total of  $5n$  parameters. Now we add some constraints at the boundaries  $[\tau_i, \tau_{i+1}]$ : in particular we require the continuity of the function and that of

its first and second derivatives. Moreover since not much information for long deliveries is present the adjustment function is less sensitive to time: hence we require that:

$$\epsilon'(\tau_n) = 0.$$

Since we have  $n + 1$  points, we have  $n - 1$  internal points in which we have to impose continuity: for  $j = 1, \dots, n - 1$  at a given  $u_j$  in the time grid we have:

$$\begin{aligned} (a_{j+1} - a_j) u_j^4 + (b_{j+1} - b_j) u_j^3 + (c_{j+1} - c_j) u_j^2 + (d_{j+1} - d_j) u_j + e_{j+1} - e_j &= 0 \\ 4(a_{j+1} - a_j) u_j^3 + 3(b_{j+1} - b_j) u_j^2 + 2(c_{j+1} - c_j) u_j + d_{j+1} - d_j &= 0, \\ 12(a_{j+1} - a_j) u_j^2 + 6(b_{j+1} - b_j) u_j + 2(c_{j+1} - c_j) &= 0. \end{aligned}$$

We have  $3(n - 1)$  constraints plus the constraint:

$$\epsilon'(\tau_n) = 4a_n u_n^3 + 3b_n u_n^2 + 2c_n u_n + d_n = 0,$$

for a total number of  $3(n - 1) + 1$  constraints so far.

Now we have to take care about the “no-arbitrage conditions” as in Equation (1). We request:

$$F(t, \tau_j^s, \tau_j^e) = \frac{1}{\tau_j^e - \tau_j^s} \int_{\tau_j^s}^{\tau_j^e} [\epsilon(u) + \Lambda(u)] du, \quad j = 1, \dots, m.$$

which translates to:

$$\int_{\tau_j^s}^{\tau_j^e} \epsilon(u) du = (\tau_j^e - \tau_j^s) F(t, \tau_j^s, \tau_j^e) - \int_{\tau_j^s}^{\tau_j^e} \Lambda(u) du. \quad (3)$$

The left-hand side integral can be directly solved, since we have the expression for  $\epsilon(u)$ , which is a forth-degree polynomial. The right-hand side integral can be easily approximated introducing a  $n$ -points grid on  $[\tau_j^s, \tau_j^e]$  obtaining:

$$\int_{\tau_j^s}^{\tau_j^e} \Lambda(u) du \approx \sum_{i=1}^N \Lambda(u_i) \Delta u_i, \quad j = 1, \dots, m.$$

Constrains in Equation (3) reduces to:

$$\begin{aligned} a_j \frac{(\tau_j^e - \tau_j^s)^5}{5} + b_j \frac{(\tau_j^e - \tau_j^s)^4}{4} + c_j \frac{(\tau_j^e - \tau_j^s)^3}{3} + d_j \frac{(\tau_j^e - \tau_j^s)^2}{2} + e_j (\tau_j^e - \tau_j^s) \\ = (\tau_j^e - \tau_j^s) F(t, \tau_j^s, \tau_j^e) - \sum_{i=1}^N \Lambda(u_i) \Delta u_i, \quad j = 1, \dots, m. \end{aligned}$$

Hence we have introduced  $m$  constraints, for a total number of  $3n + m - 2$  constraints. All these constraints are linear and hence can be expressed in matrix form  $A\mathbf{x} = \mathbf{b}$ . In the next section we show how to translate such constraints in matrix form.

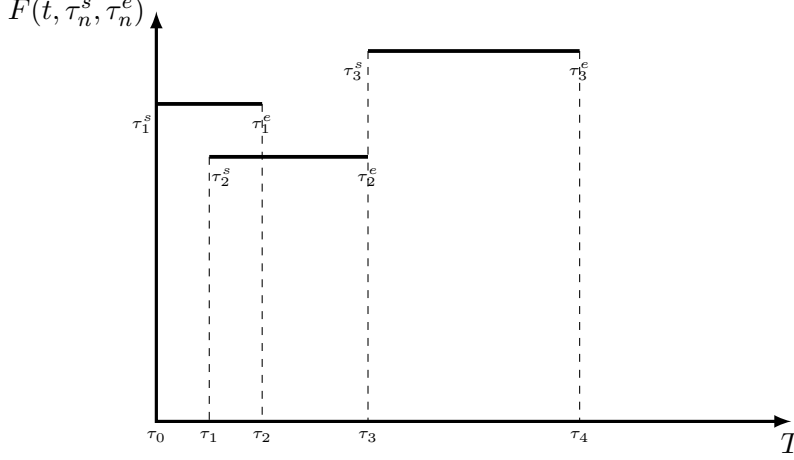


Figure 2: Simplified situation for constraints matrix construction.

### 2.2.1 The constraints matrices

Assume a very simple situation with only three products and  $\{\tau_0, \tau_1, \tau_2, \tau_3, \tau_4\}$ . So  $n = 4$  and  $m = 3$  as in Figure 2.

The vector of  $\mathbf{x} \in \mathbb{R}^{5n}$  of the unknowns is given by:

$$\mathbf{x} = [a_1, b_1, c_1, d_1, e_1, a_2, b_2, c_2, d_2, e_2, a_3, b_3, c_3, d_3, e_3, a_4, b_4, c_4, d_4, e_4].$$

Let's focus on the continuity of the second derivative at  $u_1 = \tau_1$ . From the above discussion we have:

$$12(a_2 - a_1)u_j^2 + 6(b_2 - b_1)u_1 + 2(c_2 - c_1) = 0,$$

or equivalently

$$-12a_1u_1^2 + 12a_2u_1^2 - 6b_1u_1 + 6b_2u_1 - 2c_1 + 2c_2 = 0,$$

which translated in matrix form to:

$$\begin{bmatrix} -12u_1^2, -6u_1, -2, 0, 0, 12u_1^2, 6u_1^2, 2, 0, 0, 0, \dots, 0, 0, \dots, 0 \end{bmatrix} \cdot \mathbf{x}^T = 0.$$

$\underbrace{\hspace{10em}}_{\times 5} \quad \underbrace{\hspace{10em}}_{\times 5}$

The same holds for all nodes  $j = 1, \dots, n - 1$ .

Focusing on the continuity of the function at boundaries points, at  $u_1 = \tau_1$  we have:

$$(a_2 - a_1)u_1^4 + (b_2 - b_1)u_1^3 + (c_2 - c_1)u_1^2 + (d_2 - d_1)u_1 + e_2 - e_1 = 0,$$

and in matrix form:

$$\begin{bmatrix} -u_1^4, -u_1^3, -u_1^2, -u_1, 1, u_1^4, u_1^3, u_1^2, u_1, 1, 0, \dots, 0, 0, \dots, 0 \end{bmatrix} \cdot \mathbf{x}^T = 0.$$

$\underbrace{\hspace{10em}}_{\times 5} \quad \underbrace{\hspace{10em}}_{\times 5}$

Constraints for the null derivative at  $\tau_n$  and for the continuity of the first derivative are similar and easy to obtain. For the first derivative we have:

$$\begin{bmatrix} -4u_1^3, -3b_j^2, -2c_j, -1, 0, 4u_1^3, 3b_j^2, 2c_j, 1, 0, \underbrace{0, \dots, 0}_{\times 5}, \underbrace{0, \dots, 0}_{\times 5} \end{bmatrix} \cdot \mathbf{x}^T = 0$$

Let's turn now on non-arbitrage constraints. In Figure 1 consider the second interval  $[\tau_2^s, \tau_2^e]$ . We have:

$$\tau_2^s = \tau_1 < \tau_2 < \tau_3 = \tau_2^e.$$

The non-arbitrage relation for interval  $[\tau_2^s, \tau_2^e]$  is:

$$\underbrace{\int_{\tau_2^b}^{\tau_2^e} \epsilon(u) du = (\tau_2^e - \tau_2^s) F(t, \tau_2^s, \tau_2^e) - \int_{\tau_2^b}^{\tau_2^e} \Lambda(u) du}_{\tilde{b}_2 \in \mathbb{R}}.$$

then:

$$\int_{\tau_1}^{\tau_2} (a_2^4 u^3 + b_2 u^3 + c_2 u^2 + d_2 u + e_2) du + \int_{\tau_2}^{\tau_3} (a_3^4 u^3 + b_3 u^3 + c_3 u^2 + d_3 u + e_3) du = \tilde{b}_2.$$

Defining  $\Delta_j^l = \tau_j^l - \tau_{j-1}^l$  and solving the left-hand side integral, we get:

$$\begin{bmatrix} \underbrace{0, \dots, 0}_{\times 5}, \frac{\Delta_2^5}{5}, \frac{\Delta_2^4}{4}, \frac{\Delta_2^3}{3}, \frac{\Delta_2^2}{2}, \Delta_2, \frac{\Delta_3^5}{5}, \frac{\Delta_3^4}{4}, \frac{\Delta_3^3}{3}, \frac{\Delta_3^2}{2}, \Delta_3, \underbrace{0, \dots, 0}_{\times 5} \end{bmatrix} \mathbf{x}^T = \tilde{b}_2$$

Up to now, we have three matrices:  $A_1 \in \mathbb{R}^{3(n-1) \times 5n}$  containing the constraints for the continuity of the function and of its first and second derivatives,  $A_2 \in \mathbb{R}^{1 \times 5n}$  for the constraint of the derivative at  $\tau_n$  and  $A_3 \in \mathbb{R}^{m \times 5n}$  for the non-arbitrage constraints. Associated to them we have three vectors:  $b_1 = \mathbf{0} \in \mathbb{R}^{3(n-1) \times 1}$ ,  $b_2 = 0 \in \mathbb{R}$  and  $b_3 \in \mathbb{R}^{m \times 1}$ . Recall that  $\mathbf{x}^T \in \mathbb{R}^{5n \times 1}$ . The resulting set of linear constraints is then given by:

$$A\mathbf{x}^T = \begin{bmatrix} A_1 \\ A_2 \\ A_3 \end{bmatrix} \mathbf{x}^T = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \mathbf{b}.$$

In the next section we show how to find  $\mathbf{x}$  as a solution of a proper minimization problem.

### 2.2.2 Minimization criterion

Now, we need a criterion to select the “best”  $\mathbf{x}$ , subject to  $A\mathbf{x} = \mathbf{b}$ . We minimize the roughness of the function, by solving:

$$\min_{\mathbf{x}} \int_{\tau_0}^{\tau_n} [\epsilon''(u, \mathbf{x})]^2 du,$$

subject to the constraints we discussed above. It is easy to show that such a problem can be reduced to a quadratic programming minimization problem. Indeed, focusing on a given interval  $[\tau_{j-1}, \tau_j]$ , we have:

$$\begin{aligned}\epsilon_j''(u) &= 12a_j u^2 + 6b_j u + 2c_j, \\ [\epsilon_j''(u)]^2 &= 144a_j^2 u^4 + 36b_j^2 u^2 + 4c_j^2 + 144a_j b_j u + 48a_j c_j u^2 + 24b_j c_j u. \\ \int_{\tau_{j-1}}^{\tau_j} [\epsilon_j''(u)]^2 du &= \frac{144}{5} a_j^2 u^5 + 12b_j^2 u^3 + 4c_j^2 u + 36a_j b_j u^4 + 16a_j c_j u^3 + 12b_j c_j u^2 \Big|_{\tau_{j-1}}^{\tau_j}\end{aligned}$$

Recalling that we defined  $\Delta_j^l = \tau_j^l - \tau_{j-1}^l$  we have:

$$\int_{\tau_{j-1}}^{\tau_j} [\epsilon_j''(u)]^2 du = \frac{144}{5} a_j^2 \Delta_j^5 + 12b_j^2 \Delta_j^3 + 4c_j^2 \Delta_j + 36a_j b_j \Delta_j^4 + 16a_j c_j \Delta_j^3 + 12b_j c_j \Delta_j^2$$

Hence, if we consider  $\mathbf{x}_j^T = [a_j, b_j, c_j, d_j, e_j]$  and

$$H_j = \begin{bmatrix} \frac{144}{5} \Delta_j^5 & 18\Delta_j^4 & 8\Delta_j^3 & 0 & 0 \\ 18\Delta_j^4 & 12\Delta_j^3 & 6\Delta_j^2 & 0 & 0 \\ 8\Delta_j^3 & 6\Delta_j^2 & 4\Delta_j & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

then:

$$\min_{\mathbf{x}_j} \int_{\tau_{j-1}}^{\tau_j} [\epsilon_j''(u)]^2 du = \min_{\mathbf{x}_j} \mathbf{x}_j^T H_j \mathbf{x}_j$$

By constructing a block diagonal matrix  $H$  as:

$$H = \begin{bmatrix} H_1 & 0 & \dots & \dots & 0 \\ 0 & H_2 & 0 & \dots & 0 \\ \vdots & 0 & \ddots & \dots & \vdots \\ 0 & \dots & \dots & 0 & H_n \end{bmatrix}$$

the final problem to solve is:

$$\min_{\mathbf{x}} \mathbf{x}^T H \mathbf{x}, \quad \text{s.t. } A\mathbf{x}^T = \mathbf{b}.$$

How to solve it in a easy way? Write the Lagrangian  $L(\mathbf{x}, \boldsymbol{\lambda})$ :

$$L(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{x}^T H \mathbf{x} + \boldsymbol{\lambda}^T (A\mathbf{x}^T - \mathbf{b})$$

Then:

$$\begin{aligned}\nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}) &= 2H\mathbf{x}^T + A^T \boldsymbol{\lambda} = \mathbf{0} \\ \nabla_{\boldsymbol{\lambda}} L(\mathbf{x}, \boldsymbol{\lambda}) &= A\mathbf{x}^T - \mathbf{b} = \mathbf{0}.\end{aligned}$$

which can be written in matrix form as:

$$\begin{bmatrix} 2H & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{b} \end{bmatrix} \quad (4)$$

Solving the linear system in Equation (4) we obtain  $\mathbf{x}$  and then  $\epsilon(u)$  is completely determined.

In Figure 3 we show the application of the algorithm in case in which  $\Lambda(u) = 0$ . The implementation of the code can be found in Section 5.

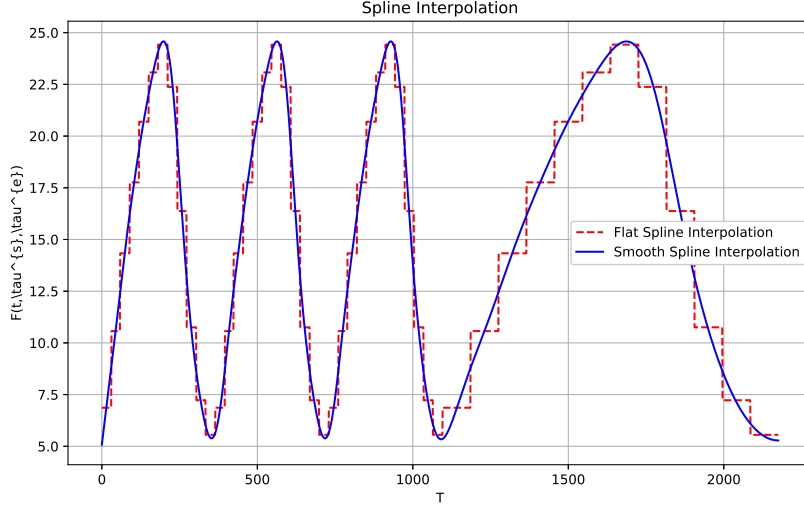


Figure 3: Example of the algorithm applied to a toy example with  $\Lambda(u) = 0$

### 2.3 Additive forward curve

Once that  $\epsilon(u)$  has been computed, since  $\Lambda(u)$  is an input the final hourly forward curve can be easily computed by:

$$f(u) = \epsilon(u) + \Lambda(u).$$

### 2.4 Multiplicative forward curve

Instead of using an additive approach we can use a multiplicative one using  $g(x, y) = xy$ . This enforces the form of the hourly forward curve to be:

$$f(u) = \epsilon(u)\Lambda(u) \tag{5}$$

One might be tempted to use exactly the same algorithm as before after introducing the transformation:

$$\ln f(u) = \ln \epsilon(u) + \ln \Lambda(u)$$

but this is not possible, since  $\Lambda(u)$  (and  $f(t)$ ) itself can be negative. Hence, we propose an easier approach simplifies things dramatically. The idea is simple: construct a smooth forward curve assuming no seasonality  $\Lambda(u)$  using the previous algorithm and then apply the hourly shape to the just constructed smooth curve.

- Assume you have an evenly spaced vector of hours,  $(t_0, \dots, t_H)$  and assume you observe prices  $\Lambda(t_h)$  for  $h = 1, \dots, H$  on your time horizon  $[\tau_0, \tau_N]$ <sup>1</sup>.
- You also have a time grid of days  $(d_0, \dots, d_K)$  where clearly  ${}^2K = \frac{H}{24}$ .

<sup>1</sup>See Section 3.1 for a discussion of future seasonality  $\Lambda(u)$ . So far, assume that  $\Lambda(u)$  is given.

<sup>2</sup>Recall that when working with LTS it is always to model in UTC format and hence localize to the right timezone, instead of modeling directly in the right timezone.

- Starting from  $\epsilon(u)$  you can compute the daily average price:

$$\epsilon(d_k) = \frac{1}{d_k - d_{k-1}} \int_{d_{k-1}}^{d_k} \epsilon(u) du, \quad k = 1, \dots, K.$$

- Now we can compute the hourly index shape  $I_{d_k}(h)$  for day  $k$  at hour  $h$ :

$$I_{d_k}(h) = \frac{\Lambda(t_h)}{\sum_{t_h \in [d_k, d_{k+1})} \Lambda(t_h)} \cdot \# \{t_h \in [d_k, d_{k+1})\}$$

where  $\# \{t_h \in [d_k, d_{k+1})\}$  is the number of hours belonging to the day  $d_k$ <sup>3</sup>. By construction, this index has average equal to one.

- The hourly forward curve at hour  $u$  of day  $d_k$  is finally obtained by:

$$f(u) = I_{d_k}(h) \epsilon(d_k)$$

Such a procedure is a very common way in electricity market modeling. Similar procedures can be applied to introduce shapes to finer granularity. In the next section we discuss some tricky points in power forward curve construction.

### 3 Tricky points

In this section we focus on some possible caveats which may arise when we construct the hourly forward curve in power markets.

#### 3.1 The form of $\Lambda(t)$

In our previous discussion we have not mentioned much about the form of  $\Lambda(t)$ . As proposed by Benth et al. [1, Chapter 7] one may assume a sinusoidal form for  $\Lambda(t)$ :

$$\Lambda(t) = A + B \cos \left( \frac{2\pi}{365.25} (t + \phi) \right)$$

where  $A$ ,  $B$  and  $\phi$  must be calibrated on history by solving:

$$\min_{A, B, \phi} \sum_{t \in T} \|S(t) - \Lambda(t)\|^2$$

where  $S = \{S(t); t \in [0, T]\}$  is the time series of the hourly spot price. Clearly many sinusoids can be added to fit a better seasonality, for example, one may assume that:

$$\Lambda(t) = \beta_0 t + \beta_1 \sin \left( \frac{2\pi}{365.25} t \right) + \beta_2 \cos \left( \frac{2\pi}{365.25} t \right) + \beta_3 \sin \left( \frac{2\pi}{365.25} t \right) + \beta_4 \cos \left( \frac{4\pi}{365.25} t \right)$$

as proposed for example by Saifert and Uhrig-Homburg [2]. This approach is very simple, since one has to fit only few parameters but hourly oscillation in the electricity prices (due to daily seasonality) are hard to catch with functional form.

---

<sup>3</sup>A fancy way to write 24.

Another possible form for  $\Lambda(t)$ , always inspired by Saifert and Uhrig-Homburg [2], can be obtained using a linear regression model with variables linked to time.

$$\Lambda(t) = \sum_{k \in N_w}^K \delta_k(t) \alpha_k + \sum_{h=1}^{24} \delta_h(t) \beta_h + \sum_{w=1}^{52} \delta_w(t) \gamma_w + \phi \delta_{\text{holiday}}(t), \quad (6)$$

where  $N_w = \{mo, tu, we, th, fr, sa, su\}$  and  $\delta(u)$  is the Dirac function which is equal to one only if time  $t$  belongs to day of week  $k$ , hour  $h$ , week of year  $w$  or is holiday. This functional form for  $\Lambda(t)$  is particularly easy to fit since the problem one has to solve is a simple linear regression. On the other hand, the drawback of this approach is the the produced hourly shape appears to be very smooth, since jumps (which are typical of the power market) are cut away by linear regression.

In order to avoid this drawback one could use a hourly shape which coming from a hourly fundamental model and directly use it as seasonality function  $\Lambda(u)$  when  $f(u)$  is constructed using the multiplicative model or the additive one. If no fundamental model are available, one can get an historical vector of hourly sport prices  $S = \{S(t); t \in [0, T]\}$ , project it into the future (being careful to match holidays and days of week) and assume this as seasonality function  $\Lambda(t)$ : in this case we are assuming that the future hourly seasonality is equal to the past one.

In any of the discussed cases, the component  $\epsilon(u)$  takes care of modifying the curve so that market forward quotations  $F(t, \tau_j^s, \tau_j^e)$ ,  $j = 1, \dots, m$  are matched.

### 3.2 Additive or multiplicative approach?

One could ask which approach is better, the additive or the multiplicative one. There is not a final answer, it depends on different factors. Using the additive approach can get you in trouble if the forward market is low and the seasonality  $\Lambda(t)$  presents extreme values. Since the final forward curve is:

$$f(u) = \Lambda(u) + \epsilon(u)$$

very negative prices may occur when you add  $\Lambda(u)$  to the spline  $\epsilon(u)$ . On the other hand, when the forward market is at a high price level and the seasonality function  $\Lambda(u)$  is not excessively volatile, the resulting forward curves do not exhibit large intraday hourly price fluctuations, a characteristic that is typically observed during periods of elevated market prices.

Furthermore, if a market doesn't admit negative prices, using a multiplicative approach prevents negative prices in the forward curve once one ensures that  $\Lambda(t) \geq 0$  for all  $t$ , assuming that  $\epsilon(u) = 0$ , which is the typical situation in power forward markets. Then

$$f(u) = \Lambda(u) \epsilon(u) \geq 0, \quad \forall u.$$

On the other hand, adopting a multiplicative approach may lead to unrealistic results. Suppose that, for a given day,  $\Lambda(t)$  is equal to zero for all hours except one. Then, when computing  $I_{d_k}(h)$ , all values are equal to zero except for a single hour, whose value is 24. Consequently, when reconstructing the forward curve  $f(u)$  as

$$f(u) = I_{d_k}(h) \epsilon(d_k)$$

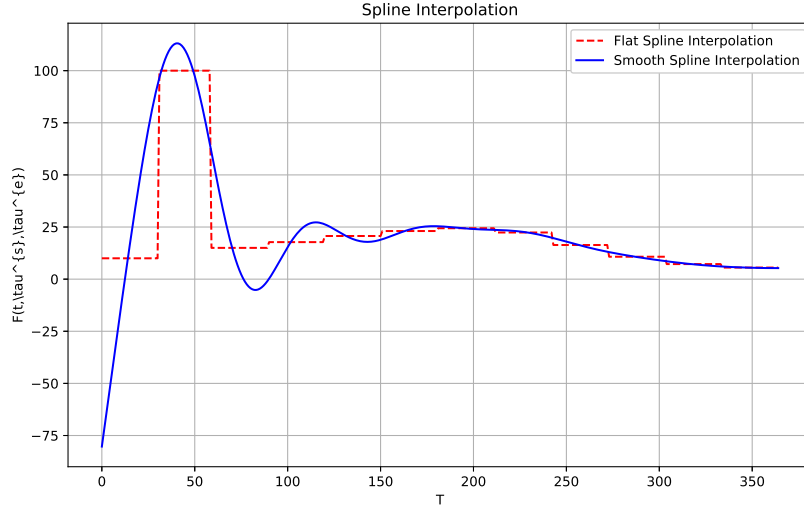


Figure 4: Oscillating splines leading to unreliable results.

all hourly values of  $f(u)$  are equal to zero except for one, which is equal to twenty-four times the average daily price  $\epsilon(d_k)$ . This clearly produces an unrealistic hourly price profile. This situation does not verify if one use the additive procedure.

### 3.3 Splines and oscillations

Splines are a powerful interpolation tool, but they may suffer from numerical issues in certain situations.

A common problem arises when two consecutive forward products,  $F(t, \tau_j^s, \tau_j^e)$  and  $F(t, \tau_{j+1}^s, \tau_{j+1}^e)$ , have significantly different prices and the interval  $[\tau_j^s, \tau_j^e]$  is short, as may occur when daily or weekly forward contracts are included. In such cases, a fourth-degree spline may exhibit large oscillations, leading to unreliable interpolation results, as illustrated in Figure 4. A possible remedy is to exclude these products from the interpolation procedure or to reduce the degree of the spline.

### 3.4 The 15-minute forward curve

Recently, day-ahead power markets have shifted from hourly granularity to 15-minute intervals. For this reason, many energy operators require a power forward curve with 15-minute resolution. In this section, we present a simple approach based on the construction of weights derived from historical observations.

First, we identify the relevant features: month ( $m$ ), hour ( $h$ ), quarter-hour ( $q$ ), and weekend indicator ( $we$ ). We then cluster the dataset of 15-minute spot prices according to these features. For each cluster, we compute historical weights, which are subsequently applied to the hourly forward curve  $f(u)$  constructed using an approach similar to the multiplicative one described above.

Let  $f(u)$  denote the electricity forward price for hour  $u$ , obtained as the output of one of the algorithms presented above, and let  $S = (S_1(q_1), S_1(q_2), \dots, S_N(q_4))$  denote the historical dataset of quarter-hourly prices. Consider the cluster  $(m, h, q, we)$  as defined in Table 1, where:

$$\begin{aligned} m &= \{1, 2, 3, \dots, 12\}, \\ h &= \{1, 2, 3, \dots, 24\}, \\ q &= \{1, 2, 3, 4\}, \\ we &= \{0, 1\}. \end{aligned}$$

Compute the average of the price  $\bar{S}_h$ ,  $h = 1, \dots, N$ , for each hour of the data-set and hence compute the weights for  $\omega_{h,q_i}$  by:

$$\omega_{h,q_i} = \frac{S_{q_i}}{\bar{S}_h}, \quad h = 1, \dots, N, \quad i = 1, \dots, 4.$$

| $m$ | $h$ | $q$ | $w$ | $S_{q_i}$ | $\bar{S}_h$ | $\omega_{h,q_i}$ |
|-----|-----|-----|-----|-----------|-------------|------------------|
| 1   | 1   | 1   | 0   | 105.33    | 110.11      | 0.957            |
| 1   | 1   | 2   | 0   | 106.12    | 110.11      | 0.964            |
| 1   | 1   | 3   | 0   | 108.32    | 110.11      | 0.984            |
| 1   | 1   | 4   | 0   | 120.70    | 110.11      | 1.096            |
| 1   | 1   | 1   | 0   | 115.33    | 128.35      | 0.899            |
| 1   | 1   | 2   | 0   | 119.12    | 128.35      | 0.928            |
| 1   | 1   | 3   | 0   | 128.32    | 128.35      | 1.000            |
| 1   | 1   | 4   | 0   | 150.65    | 128.35      | 1.174            |
| 1   | 2   | 1   | 0   | 102.33    | 108.87      | 0.940            |
| 1   | 2   | 2   | 0   | 102.12    | 108.87      | 0.938            |
| 1   | 2   | 3   | 0   | 105.32    | 108.87      | 0.967            |
| 1   | 2   | 4   | 0   | 125.70    | 108.87      | 1.155            |
| ... | ... | ... | ... | ...       | ...         | ...              |
| 1   | 1   | 1   | 1   | 205.33    | 221.61      | 0.927            |
| 1   | 1   | 2   | 1   | 256.12    | 221.61      | 1.156            |
| 1   | 1   | 3   | 1   | 188.32    | 221.61      | 0.850            |
| 1   | 1   | 4   | 1   | 236.70    | 221.61      | 1.068            |

Table 1: Table with weights for the quarterly hour curve.

As final step, we get the average of column  $\omega_{h,q_i}$  for each unique key  $(m, h, q, w)$ :

$$\omega_{m,h,q,w} = \frac{1}{\#\{(m, h, q, w)\}} \sum_{\omega_{h,q_i} \in \{(m, h, q, w)\}} \omega_{h,q_i},$$

for  $m = \{1, \dots, 12\}$ ,  $h = \{1, \dots, 24\}$ ,  $q = \{1, \dots, 4\}$ ,  $we = \{0, 1\}$ .

Finally, the price for the quarter-hour interval  $q_i$  in the constructed forward curve is given by:

$$f(u, q) = f(u) \cdot \omega_{m,h,q,w}, \quad q = 1, \dots, 4.$$

By construction, all no-arbitrage constraints are satisfied, and the following consistency condition holds:

$$\frac{1}{4} \sum_q^4 f(u, q) = f(u), \quad \forall u.$$

In the next section we show some results obtained by the proposed algorithms.

## 4 Results

In this section, we present several plots of forward curves constructed using the different methods described above. In the *linear* approach, we model the seasonality term  $\Lambda(t)$  using a linear specification of the form given in Equation (6). In contrast, the *historical* approach relies on past observations, which are projected into the future by matching months, day of the week, and holidays.

To construct forward curves with 15-minute granularity, we use the procedure described in Section 3.4.

- In both Figure 5 and Figure 6, we observe that the additive curve may produce negative values, in contrast to the multiplicative one.
- Comparing Figure 5 with Figure 6, we find that the linear method yields smoother curves compared to the historical one. This is because, in the linear approach, seasonality is estimated using a linear regression model, whereas in the historical approach it is directly derived from historical data, which may contain spikes.
- In Figures 7 and 8, we apply the refinement to 15-minute granularity described above and compare the resulting curves with the corresponding hourly forward curves obtained using the linear and historical seasonality methods, respectively.

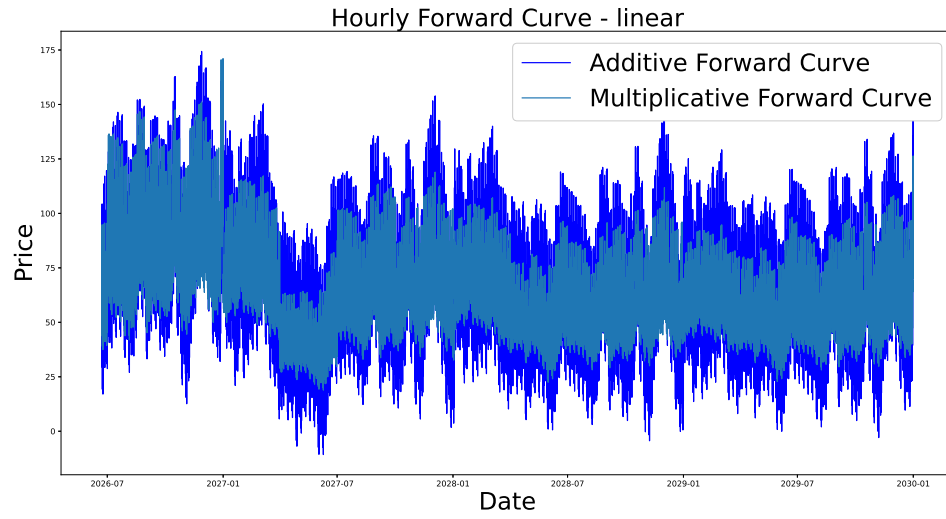


Figure 5: Linear method for seasonality  $\Lambda(t)$ : additive forward curve vs multiplicative forward curve.

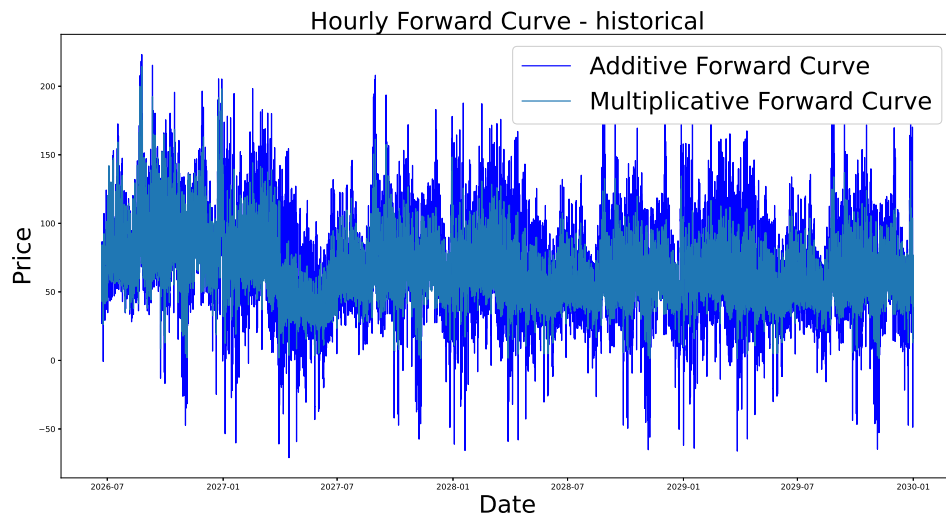


Figure 6: Historical method for seasonality  $\Lambda(t)$ : additive forward curve vs multiplicative forward curve.

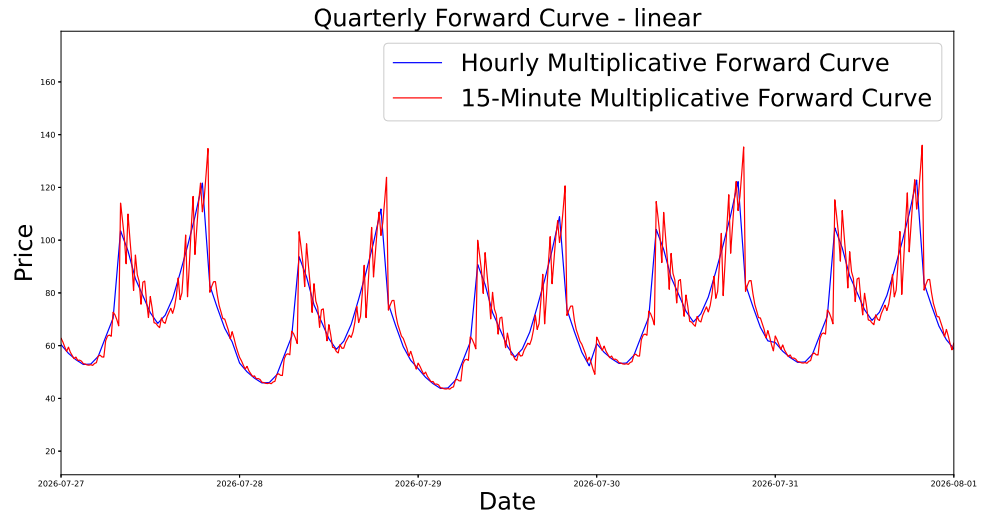


Figure 7: Linear method for seasonality  $\Lambda(t)$ : fifteen minutes forward curve vs hourly forward curve.

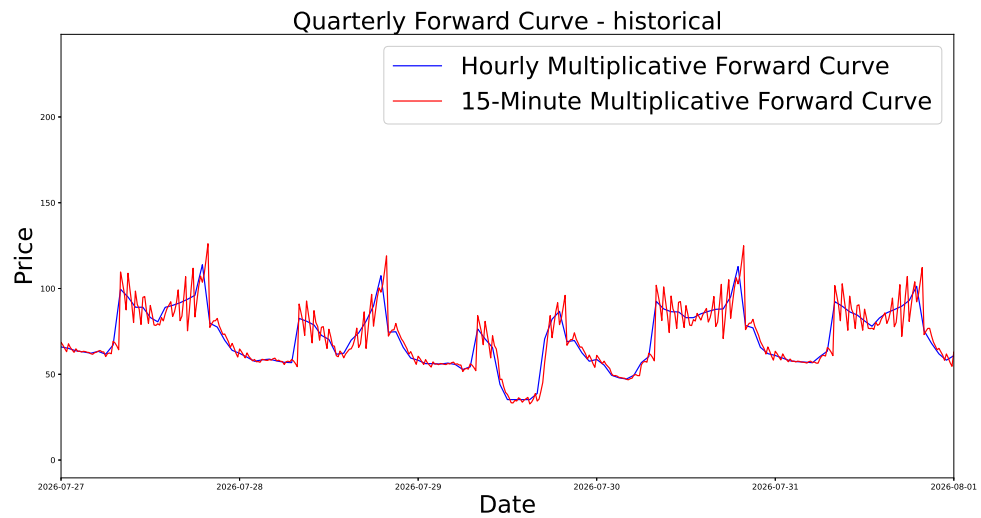


Figure 8: Historical method for seasonality  $\Lambda(t)$ : fifteen minutes forward curve vs hourly forward curve.

## 5 Splines\_spliner.py module

This module contains the implementation of the interpolation method.

```
1 import numpy as np
2 import pandas as pd
3 from enum import StrEnum
4
5 class InterpolatorFieldName(StrEnum):
6     tau_s = "tau_s"
7     tau_e = "tau_e"
8     val = "value"
9
10
11 class SplineInterpolatorConstrained:
12     def __init__(self, df: pd.DataFrame):
13         self.df_prices = df
14
15         self.tau_b = np.array(df[InterpolatorFieldName.tau_s])
16         self.tau_e = np.array(df[InterpolatorFieldName.tau_e])
17         self.p = np.array(df[InterpolatorFieldName.val])
18
19         self.flat_curve = None
20         self.smooth_curve = None
21
22
23     def create_A1_matrix(self) -> np.ndarray:
24         """
25         Create the A1 matrix for the spline interpolation problem.
26
27         Returns
28         -----
29         A1 : np.ndarray
30             The A1 matrix for the spline interpolation problem.
31
32         """
33
34         # constraints
35         # You have one constraint for each interval.
36         c1 = np.zeros((self.n - 1, 5 * self.n))
37         c2 = np.zeros((self.n - 1, 5 * self.n))
38         c3 = np.zeros((self.n - 1, 5 * self.n))
39         # You have the constrain for the derivative
40         c4 = np.zeros(5 * self.n)
41
42         # The following constraints holds only for internal nodes (number of
43         # nodes-2)
44         # n is the number of intervals
45         for j in range(self.n - 1):
46             # the positive values start from column i+1
47             # Remember python excludes the right most index
48             c1[j, 5 * (j + 1):5 * (j + 1) + 5] = np.array(
```

```

48         [self.tau[j + 1] ** 4.0, self.tau[j + 1] ** 3.0, self.tau[j + 1]
** 2.0, self.tau[j + 1], 1])
49         c1[j, 5 * j:5 * (j + 1)] = -c1[j, 5 * (j + 1):5 * (j + 1) + 5]
50
51         c2[j, 5 * (j + 1):5 * (j + 1) + 5] = np.array(
52             [4.0 * self.tau[j + 1] ** 3.0, 3.0 * self.tau[j + 1] ** 2.0, 2.0
* self.tau[j + 1], 1, 0])
53         c2[j, 5 * j:5 * (j + 1)] = -c2[j, 5 * (j + 1):5 * (j + 1) + 5]
54
55         c3[j, 5 * (j + 1):5 * (j + 1) + 5] = np.array([12.0 * self.tau[j + 1]
** 2.0, 6.0 * self.tau[j + 1],
56                                                         2.0, 0, 0])
57         c3[j, 5 * j:5 * (j + 1)] = -c3[j, 5 * (j + 1):5 * (j + 1) + 5]
58
59         # Constrain on the derivative
60         c4[5 * self.n - 5] = 4.0 * self.tau[-1] ** 3.0
61         c4[5 * self.n - 4] = 3.0 * self.tau[-1] ** 2.0
62         c4[5 * self.n - 3] = 2.0 * self.tau[-1]
63         c4[5 * self.n - 2] = 1
64
65         # Build the matrix A1
66         A1 = np.vstack((c1, c2, c3, c4))
67
68         return A1
69
70     def create_H_matrix(self)->np.ndarray:
71         """
72         Create the H matrix for the spline interpolation problem. The H matrix is
73         used to enforce smoothness constraints on the spline.
74         """
75
76
77         # Now you have to compute the matrix H containing the minimization
78         # constraints
79         H = np.zeros((5 * self.n, 5 * self.n))
80
81         for j in range(1, self.n + 1):
82             h = np.zeros((5, 5))
83             h[0, 1] = 18 * (self.tau[j] ** 4.0 - self.tau[j - 1] ** 4.0)
84             h[0, 2] = 8 * (self.tau[j] ** 3.0 - self.tau[j - 1] ** 3.0)
85             h[1, 2] = 6 * (self.tau[j] ** 2.0 - self.tau[j - 1] ** 2.0)
86             h = h + h.transpose()
87
88             h[0, 0] = 144.0 / 5.0 * (self.tau[j] ** 5.0 - self.tau[j - 1] ** 5.0)
89             h[1, 1] = 12 * (self.tau[j] ** 3.0 - self.tau[j - 1] ** 3.0)
90             h[2, 2] = 4 * (self.tau[j] - self.tau[j - 1])
91
92             H[(j - 1) * 5:j * 5, (j - 1) * 5:j * 5] = h
93
94         return H

```

```

95
96 def create_known_terms_vector(self)->tuple[np.ndarray, np.ndarray, np.ndarray
97 ]:
98     """
99     Create the known terms vector for the spline interpolation problem.
100
101     Returns
102     -----
103     A2 : np.ndarray
104         The A2 matrix for the spline interpolation problem.
105     b : np.ndarray
106         The known terms vector for the spline interpolation problem.
107     b_flat : np.ndarray
108         The known terms vector for the flat forward curve.
109     A_flat : np.ndarray
110         The A_flat matrix for the flat forward curve.
111     """
112
113     # A2 matrix
114     A2 = np.zeros((self.m, 5 * self.n))
115
116     # Known terms vector
117     b = np.zeros((3 * self.n + self.m - 2, 1))
118
119
120     # Known terms vector: these are the elements needed to compute the flat
121     forward curve
122     b_flat = np.zeros((self.m, 1))
123     A_flat = np.zeros((self.m, self.n))
124
125     # Integral constraints
126     for i in range(self.m):
127         # supp_int = (u >= tau_b[i]) & (u < tau_e[i])
128         # integral_term = np.sum(shape[supp_int]) * (u[2] - u[1]) # Assume
129         an equispaced shape
130         b[3 * self.n - 2 + i, 0] = self.p[i] * (self.tau_e[i] - self.tau_b[i
131 ]) # - integral_term # Known term
132
133         b_flat[i] = self.p[i] * (self.tau_e[i] - self.tau_b[i]) # Known term
134
135     # Loop over the intervals
136     for j in range(self.n):
137         if (self.tau[j] >= self.tau_b[i]) & (self.tau[j] < self.tau_e[i])
138
139         :
140
141         A2[i, j * 5:(j + 1) * 5] = np.array \
142             ([ (self.tau[j + 1] ** 5.0 - self.tau[j] ** 5.0) / 5.0, \
143               (self.tau[j + 1] ** 4.0 - self.tau[j] ** 4.0) / 4.0, \
144               (self.tau[j + 1] ** 3.0 - self.tau[j] ** 3.0) / 3.0, \
145               (self.tau[j + 1] ** 2.0 - self.tau[j] ** 2.0) / 2.0, \
146               (self.tau[j + 1] - self.tau[j]))
147         # A_flat[i, j] = np.array([(tau[j + 1] - tau[j])])

```

```

142         A_flat[i, j] = self.tau[j + 1] - self.tau[j]
143
144     return A2, b, b_flat, A_flat
145
146     def solve_linear_systems(self, A_flat: np.ndarray, b_flat: np.ndarray, M: np.
147     ndarray, y: np.ndarray) -> np.ndarray:
148         """ Solve both the flat and smooth forward curve linear systems.
149
150         Parameters
151         -----
152         A_flat : np.ndarray
153             The A_flat matrix for the flat forward curve.
154         b_flat : np.ndarray
155             The known terms vector for the flat forward curve.
156
157         M : np.ndarray
158             The M matrix for the smooth forward curve.
159         y : np.ndarray
160             The known terms vector for the smooth forward curve.
161
162         """
163
164         # This is the case when you want to compute the flat forward curve
165         x_flat = np.linalg.solve(A_flat, b_flat)
166         # select the coefficients
167         coeff_flat = np.zeros([self.n, 1])
168         for i in range(self.n):
169             coeff_flat[i, 0] = x_flat[i, 0]
170
171         # This is the case when you want to compute the smooth forward curve
172         x_ = np.linalg.solve(M, y)
173         # select the coefficients
174         coeff = np.zeros([self.n, 5])
175         for i in range(self.n):
176             coeff[i, 0] = x_[5 * i, 0]
177             coeff[i, 1] = x_[5 * i + 1, 0]
178             coeff[i, 2] = x_[5 * i + 2, 0]
179             coeff[i, 3] = x_[5 * i + 3, 0]
180             coeff[i, 4] = x_[5 * i + 4, 0]
181
182         return coeff, coeff_flat
183
184     def spline_interpolator(self)->pd.Series:
185         """
186         Construct the forward curve.
187
188         Parameters
189         -----
190         flat_forward_curve : bool
191             If True, the forward curve is constructed as flat, that is, without
192             seasonality adjustments or spline smoothing.

```

```

193     Returns
194     -----
195     pd.Series
196         The constructed curve as a pandas Series.
197     """
198
199     # Take only the unique taus
200     self.tau = np.unique(np.hstack((self.tau_b, self.tau_e)))
201
202     u = np.linspace(0, int(self.tau[-1]) - 1, int(self.tau[-1]))
203
204     self.n = len(self.tau) - 1 # Number of intervals
205     self.m = len(self.p)      # Number of values
206
207     # Create the matrices you need to solve the problem
208     A1 = self.create_A1_matrix()
209     A2, b, b_flat, A_flat = self.create_known_terms_vector()
210
211     # Concatenate A1 and A2
212     A = np.vstack([A1, A2])
213     AT = A.transpose()
214
215     Z = np.zeros((3 * self.n + self.m - 2, 3 * self.n + self.m - 2))
216
217     # Now you have to compute the matrix H containing the minimization
constraints
218     H = self.create_H_matrix()
219
220     # Stack the matrices together in one common matrix.
221     M1 = np.hstack([2 * H, AT])
222     M2 = np.hstack([A, Z])
223     M = np.vstack([M1, M2])
224
225     # Create the known terms vector for the linear system
226     y = np.zeros((5 * self.n + 3 * self.n + self.m - 2, 1))
227     y[5 * self.n:] = b
228
229     # Solve the linear system:
230     coeff, coeff_flat = self.solve_linear_systems(A_flat, b_flat, M, y)
231
232
233     # Pre-allocation
234     epsilon = np.zeros(len(u))
235     epsilon_flat = np.zeros(len(u))
236
237     # Fill epsilon evaluating the polynomial
238     for i in range(self.n):
239         supp_temp = (u >= self.tau[i]) & (u < self.tau[i + 1])
240         epsilon[supp_temp] = np.polyval(coeff[i, :], u[supp_temp] + 1)
241         epsilon_flat[supp_temp] = np.polyval(coeff_flat[i, :], u[supp_temp] +
1)
242

```

```
243     epsilon[-1] = epsilon[-2]
244
245     self.smooth_curve = epsilon
246     self.flat_curve = epsilon_flat
247
248     return epsilon, epsilon_flat
249
250
251
252 __all__ = ["SplineInterpolatorConstrained"]
```

## References

- [1] F.E. Benth, J.S. Benth, and S. Koekebakker. *Stochastic Modeling of Electricity and Related Markets*. Number 6811 in World Scientific Books. World Scientific Publishing Co. Pte. Ltd., July 2008. ISBN ARRAY(0x50724998). URL <https://ideas.repec.org/b/wsi/wsbook/6811.html>.
- [2] J. Saifert and M. Uhrig-Homburg. *Modelling Jumps in Electricity Prices: Theory and Empirical Evidence*. 2006.